Ira A. Fulton Schools of Engineering
Arizona State University

CSE 568: Biocomputing

Extensions to Balanced Colorings and Neutral Landscapes Project

Purpose

The purpose of this project is to examine balanced graph coloring and neutral coloring landscapes through a genetic-algorithm-based optimization problem. You will refine a graph-coloring fitness function by adding a balance requirement and then analyze how neutral mutations affect the search landscape. This work gives you practice designing fitness functions that capture multiple goals, which is important in computational biology, optimization, and other industry applications where practical solutions often require balancing competing constraints.

Objectives

Learners will be able to:

- **2.1:** Describe the biological foundations of evolutionary computation.
- **2.2:** Apply evolutionary computation algorithms to solve optimization problems.
- **2.3:** Implement a genetic algorithm to solve graph-coloring problems.
- **3.2:** Explain how evolutionary algorithms search high-dimensional spaces using biased sampling.
- **3.3:** Refine a graph-coloring model using evolutionary algorithms.

Technology Requirements

- A local Python development environment
- Python 3

- Required Python packages:
  - deap
  - matplotlib
  - numpy
- Access to Gradescope

Description

In this project, you will reuse and extend the graph-coloring fitness function developed earlier in the course to evaluate balanced graph colorings. A balanced coloring accounts for whether adjacent vertices use different colors and whether the available colors are distributed evenly. You will refine the fitness function by adding a balance requirement and compute one-step neutrality by measuring how many single-color mutations preserve the original coloring's fitness. As you evaluate balanced colorings and their mutations, you will explore a more complex graph-coloring search problem and analyze how fitness, mutation, and neutrality shape evolutionary search behavior.

The project has three main goals:

- Use a genetic-algorithm-style fitness function to evaluate balanced graph colorings.
- Reuse your Project 01 graph-coloring fitness function.
- Compute the one-step neutrality of a graph coloring.

Directions

Before getting started, refer to the additional resources and files provided in the course to complete your project accurately:

- Project Overview Video
- Project ZIP File

Graph Representation

```
graph.n       # number of vertices
graph.k       # number of valid colors
graph.graph   # adjacency-list heads; each node has .vertex and .next
```

The graph is undirected. Each edge is stored twice in the adjacency lists, once from u to v and once from v to u. When computing fitness, count each undirected edge only once.

Individual Encoding

Use the same bitstring encoding from Project 01:

```
bits_per_vertex = ceil(log2(graph.k))
```

Each vertex color is decoded from one consecutive chunk of bits, most-significant bit first. For graph.k == 3, each vertex uses two bits:

```
00 -> color 0
01 -> color 1
10 -> color 2
11 -> invalid color encoding
```

If any decoded color is outside the valid range 0 through graph.k - 1, the coloring fitness should be 0.0.

Question 1: Balanced-coloring fitness

Implement:

```
def eval_balance(graph, individual):
    ...
```

A balanced coloring is a coloring where adjacent vertices should not share the same color and the available colors should be used evenly.

Let G = (V, E) be a graph with n vertices, m undirected edges, and k colors. Let c(i) be the color assigned to vertex i. For an edge (i, j), define delta(i, j) = 1 if c(i) != c(j), and delta(i, j) = 0 otherwise.

The balanced-coloring fitness is:

```
F(C(G)) = (sum over edges delta(i, j) / m) * product over colors c of (count(c) / n)
```

The product should include every valid color from 0 through graph.k - 1. If a color is missing, its factor is zero, so the total balanced-coloring fitness is zero. Return the result as a one-item tuple:

```
return (fitness,)
```

Sample for graphs/graph_1.txt:

```
individual = [0, 1, 0, 0, 1, 0, 0, 1]
# decoded colors: [1, 0, 2, 1]
eval_balance(graph, individual) == (0.03125,)
```

Question 2: Project 01 Graph-coloring Fitness Helper

Implement:

```
def eval_graph(graph, individual):
    ...
```

This function is the standard graph-coloring fitness from Project 01. It should compute the fraction of undirected edges whose endpoints have different colors:

```
fitness = satisfied_edges / total_edges
```

Return a one-item tuple, (fitness,). If any decoded color is invalid, return (0.0,). Question 2 is not assigned a separate score in the Project 02 Gradescope tests, but one_neutral depends on this function or equivalent logic.

Question 3: One-step Neutrality

Implement:

```
def one_neutral(graph, individual):
    ...
```

A mutation is neutral if it does not change the graph-coloring fitness of an individual. In this question, calculate one-step neutrality by changing one vertex color at a time.

1. Decode the input individual into one color per vertex.
2. Compute the original coloring fitness using eval_graph.
3. For each vertex, create mutants by replacing that vertex color with every other valid color.
4. Evaluate each mutant using eval_graph.
5. Count how many mutants have the same fitness as the original coloring.
6. Return neutral_count / total_mutations as a float.

There are graph.n * (graph.k - 1) one-step color mutants. The code sample for graphs/graph_1.txt should return:

```
one_neutral(graph, [0, 1, 0, 0, 1, 0, 0, 1]) == 0.25
```

Submission Directions for Deliverables

You must submit your Extensions to Balanced Colorings and Neutral Landscapes Project deliverable through **Gradescope**. Carefully review submission directions outlined in this overview document in order to correctly earn credit for your work. Learners may not email or use other means to submit any assignment or project for review, including feedback and grading.

The Extensions to Balanced Colorings and Neutral Landscapes Project includes one (1) deliverable:

- **student_solution.py:** Submit one (1) Python solution. Do not submit the autograder zip or extra output files. You may add helper functions, but do not rename the required functions.

```
def eval_balance(graph, individual):
    ...
```

```
def eval_graph(graph, individual):
    ...

def one_neutral(graph, individual):
    ...
```

Gradescope Submission

Your submission will be reviewed by the course team, and then, after the due date has passed, your score will be populated from Gradescope into your grade.

1. Go to the Canvas Assignment, “**Submission: Extensions to Balanced Colorings and Neutral Landscapes Project**”.
2. Click the "**Load Submission…in new window**" button.
3. Once in Gradescope, select the project titled "**Extensions to Balanced Colorings and Neutral Landscapes Project,**" and a pop-up window will appear.
4. In the pop-up window, submit your **“student_solution.py**” file for grading.
5. If needed: to resubmit the project in Gradescope:
    a. Return to the Canvas submission and open Gradescope.
    b. You will be navigated to the "Autograder Results" page (if it is not your first submission).
    c. Click the "**Resubmit**" button on the bottom right corner of the page and repeat the process from Step 3.

Evaluation

Please review the table for how your project will be graded. The project deliverable will be evaluated by the autograder based on each criterion and will receive a total score.

Deliverables missing any part of the project will be graded based on what was submitted against the grading criteria. Missing parts submitted after the deadline will not be graded.

Review the course syllabus for details regarding late penalties.

Gradescope Weights

The autograder awards 100 total points.

Questions	Points
Q1: eval_balance sample	15
Q1: eval_balance formula on another graph	15
Q1: invalid color encoding returns zero	10
Q1: missing color bucket returns zero	10
Q3: one_neutral sample	20
Q3: one_neutral on another graph	15
Q3: one_neutral on a larger graph	15
Total	**100**